

Healthcare Customer Support Router Agentic RAG — Project Explanation

Overview

This project builds a Router Agentic RAG system for a healthcare provider's customer support using LangGraph, OpenAI and ChromaDB. Instead of answering every message the same way, the system first decides how a query should be handled, then either answers it from the right part of the knowledge base or escalates it to a person.

What the Notebook Does

A healthcare knowledge base is loaded from JSON, converted into LangChain documents that keep a department category in their metadata, and embedded into a persistent Chroma vector store using cosine similarity. A score-threshold retriever returns no results when nothing is relevant, and the notebook explicitly demonstrates that metadata filters are case-sensitive. A typed agent state tracks the query, its category, its sentiment, any form details and the final response, while two Pydantic models with literal types force the language model to return only valid department and sentiment labels.

Seven LangGraph nodes then process each query. The first two classify it by department and by sentiment. A routing function checks sentiment before category: negative queries go to an escalation form and a human support agent, distress queries go to an emergency form and the on-call medical team, and positive or neutral queries go to a department-filtered RAG node that answers only from retrieved knowledge and falls back to a contact message when nothing relevant is found. The forms use ipywidgets with jupyter-ui-poll, the graph is compiled with an in-memory checkpoint so each customer session keeps its own state, and seven test queries confirm every route behaves as intended.

Skills Demonstrated

- Designing conditional routing in a LangGraph state machine
- Structured LLM output with Pydantic literal types for reliable classification
- Metadata-filtered, score-threshold retrieval with ChromaDB
- Human-in-the-loop escalation paths for sensitive, high-risk queries

The pattern of classifying, routing and escalating before generating is directly relevant to my dissertation's agentic intelligence platform, where different query types must reach different specialised agents.